

Project Application of Quantum-RegeV-Algorithm Inside Real Quantum Machines For IQM Challenge Award

Engineer: Alisandro Nadal

The Project is inside GitHub Repository: <https://github.com/threealgos/Quantum-RegeV-Cracker>

i Made A Successful Achievements inside Your Platform :_ **IQM_Resonance Q-Machines (Emerald, Garnet, Sirius)**

i Programed an effective Quantum_Code That Based on Implementation of The New Regev's Quantum Factorization Algorithm Accelerated in Multiple Dimensions Using Regev_Lattices That was Theoretically Suggested By Professeur Oded Regev : <https://www.youtube.com/shorts/cc9iBROsNH8>

Executive Summary

Quantum-RegeV-Cracker explores Regev-inspired quantum techniques on real quantum hardware. Experiments were executed on IQM Resonance systems and compared against IBM Quantum platforms. The project evaluates hybrid quantum-classical workflows, measurement aggregation methods, and lattice-inspired processing under NISQ-era constraints while remaining compatible with future fault-tolerant systems.

Background & Architecture

Development evolved from earlier Quantum KeySpace Reducer research. The workflow combines quantum circuit generation, repeated hardware execution, measurement collection, aggregation, classical reduction procedures, and cryptographic validation. Implementations integrate Qiskit, IQM tooling, qBraid workflows, and custom validation modules.

Benchmark Results

Benchmark	Shots	Reported Result
16-bit	~2,048	Successful demonstration
19-bit	16,384	~99.9% success
20-bit	16,384	~50% success under noisier conditions

Keyspace Context

19-bit benchmark: 262,144 candidate private keys. 20-bit benchmark: 524,288 candidate private keys. These small search spaces were used for controlled experimental evaluation.

Cross-Platform Validation & IQM Challenge Relevance

Testing on both IQM and IBM hardware demonstrated that results were not dependent on a single backend calibration profile. The project was submitted as an IQM Challenge entry emphasizing real hardware execution, reproducibility, and scalability under realistic hardware constraints.

Future Roadmap & Significance

The architecture is designed for future fault-tolerant processors. As logical qubits, error correction, and gate fidelities improve, deeper circuits and larger problem instances can be evaluated without fundamental redesign. The project contributes practical observations on shot budgets, noise behavior, backend stability, and hybrid quantum-classical computation.

Conclusion

Quantum-Regev-Cracker serves as a proof-of-concept research effort demonstrating execution on real quantum hardware, cross-platform validation, and Regev-inspired processing techniques. It establishes a foundation for future investigations into larger-scale quantum cryptographic applications as quantum technology matures.

Recommended Configuration :

QUANTUM-REGEV-1920 – RECOMMENDED CONFIG (DEFAULT)			
THIS IS BEST FAST CONFIG TO CRACK SMALL BIT KEYS: REGEV + IPE (RECOMMENDED CONFIGS)			
START PROGRAM WITH: <code>python3 Quantum-Regev-1920.py</code>		FOR COSTUME WRITE: C	
1) PASTE COMPRESSED PUBLIC KEY 03 XX 02 XX		2) INPUT YOUR TARGET BIT LENGTH Input Your Target bit length and your exact bit-length (or 16 / 17 / 18 / 19 / 20 /135) example input: 19 Just Press Enter for [auto] k_start (hex) [auto]: [Press Enter]	
3) NUMBER OF SHOTS FOR ACCURATE RESULT Write: 16384			
4) CHOOSE SOLVER [1] Regev + IPE Hybrid (default — recommended) Choose: 1 (Regev + IPE)		5) SOLVER OPTIONS (EXACTLY THIS) ☒ Disable --> HalfGCD modular inversion? [Y/n]: n ☒ Disable --> Measurement-based uncomputation (MBU)? [Y/n]: n ☒ Enable --> Fibonacci basis-point prep (Ragavan-VV)? [Y/n]: Y --> Noise-filter sigma (Ragavan-VV, 0=off) [2.0]: 2.0	
6) ADDER Select [adder] choose ---> : approx Approx threshold [4] Choose: 4			
7) ERROR ENCODING (Regev / Regev+IPE only) [none] No encoding [repetition] [[3,1,1]] bit-flip code [surface] Surface-d3 patch (single round, decorative) [cat] Cat-qubit approximation [dualrail] Dual-rail erasure detection Select: cat		8) MISC OPTIONS Enable --> Clifford+T optimization? [Y/n]: Y Disable --> Enable flag qubits? [Y/n]: n	
9) SDK [qiskit] Qiskit (default) [pytket] pytket Select [qiskit/pytket]: pytket Select [qiskit/pytket]: qiskit (JUST Write The One you want & press ENTER)			
10) BACKENDS: BEST MACHINES WORKED FINE (IBM / IQM) – DON'T WORRY ABOUT WARNINGS			
CHOOSE BACKEND if you choosed qiskit --> Next Write [ibm] in The Next Backends inputs. if you choosed pytket --> Next Write [iqm] in The Next Backends inputs. [aer] Aer simulator [ibm] IBM Quantum [iqm] IQM Resonance (pytket-iqm: sirius/garnet/emerald) Choose ---> Select [aer]: ibm / Select [aer]: iqm		NUMBER OF RUNS (Regev needs d+4 independent samples) Choose [1]: 1	
IBM ----- FOR IBM ----- IBM token: XXH4dAKS6t9MZC6NGKtVGi0j GL6nO_vONBAnOab5pTTo IBM CRN [optional]: [Press Enter] IBM backend name [ibm_fez]: example: ibm_kingston / ibm_fez OR You Can Choose Write Your [Backends Name] & Press ENTER.		IQM ----- FOR IQM ----- IQM token: cL7Kp6KWbIOxdwMG.LOWVRcqMRjdfw44 RxXlWzo3o554BnqZ4oBh5Qrq210cBRTbD IQM device [garnet / sirius / emerald] Write it: emerald FOR IQM IT IS DONE HERE.	
★ SUMMARY CHECKLIST			
☒ Solver: Regev + IPE ☒ HalfGCD: n ☒ Error Encoding: cat ☒ SDK: (qiskit / pytket)			
☒ MBU: n ☒ Adder: approx (4) ☒ Clifford+T: Y ☒ Backend: ibm (for qiskit) / iqm (for pytket)			
☒ Ragavan-VV: Y (sigma 2.0) ☒ Flag Qubits: n ☒ Shots: 16384 Runs: 1			
🏆 THIS CONFIG = BEST SPEED FOR SMALL BIT KEYS!			